

Módulo 3: Algoritmos de Machine Learning – No Supervisado, Refuerzo y AutoML

Material de Evaluación y Práctica

Este documento complementa el estudio de Machine Learning explorando paradigmas más allá del aprendizaje supervisado tradicional, incluyendo la automatización de modelos (AutoML) y el aprendizaje por refuerzo.

Fundamentos Teóricos: Paradigmas y Herramientas

1. Paradigmas de Aprendizaje

- **Aprendizaje Supervisado:** El modelo aprende a partir de datos etiquetados (entrada y salida conocida). Su objetivo es predecir etiquetas para nuevos datos.
- **Aprendizaje No Supervisado:** El modelo trabaja con datos **sin etiquetas**. Su objetivo es encontrar estructuras o patrones ocultos (agrupamiento o reducción de dimensiones).
- **Aprendizaje Semisupervisado:** Se sitúa entre los dos anteriores. Utiliza una pequeña cantidad de datos etiquetados combinada con una gran cantidad de datos no etiquetados para mejorar la precisión del aprendizaje.
- **Aprendizaje por Refuerzo:** Se basa en un agente que interactúa con un entorno y aprende a tomar decisiones mediante un sistema de **recompensas y castigos**. El objetivo es maximizar la recompensa acumulada.

2. Automatización y ML Moderno

- **AutoML (Automated Machine Learning):** Es el proceso de automatizar las tareas repetitivas del desarrollo de modelos (preprocesamiento, selección de algoritmos, optimización de hiperparámetros).
 - **LazyPredict:** Una librería diseñada para realizar una comparación rápida y masiva de decenas de modelos supervisados (clasificación o regresión) sobre un dataset sin necesidad de configurar cada uno manualmente.
 - **PyCaret:** Una librería de "código bajo" (*low-code*) que permite gestionar todo el ciclo de vida de un proyecto de ML (desde la preparación de datos hasta el despliegue) en muy pocas líneas de código.
-

Ejercicio 1 - Agrupamiento con K-Means

Objetivo: Aplicar el algoritmo de K-Means para segmentar clientes basándose en su comportamiento de compra.

Enunciado del Reto: Un centro comercial quiere agrupar a sus clientes según sus ingresos anuales y su puntuación de gasto. Utiliza los datos sintéticos para crear 5 grupos de clientes y visualiza los centroides de cada grupo.

Solución en Python:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# 1. Generación de datos sintéticos (Ingresos vs Gasto)
np.random.seed(42)
X = np.random.rand(200, 2) * 100

# 2. Aplicar K-Means con k=5
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X)

# 3. Visualización
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X',
            label='Centroides')
plt.title('Segmentación de Clientes (K-Means)')
plt.xlabel('Ingresos Anuales')
plt.ylabel('Score de Gasto')
plt.legend()
plt.show()
```

Resultados Esperados de la Ejecución:

- **Gráfico generado:** Se creará una imagen mostrando la distribución de los 200 puntos en 5 clústeres de diferentes colores, junto con sus respectivos centroides marcados con una 'X' roja.

Ejercicio 2 - Optimización: El Método del Codo (Elbow Method)

Objetivo: Analizar cuál es el número óptimo de clusters (\$k\$) para un conjunto de datos desconocido.

Enunciado del Reto: No siempre sabemos en cuántos grupos dividir los datos. Utiliza la métrica de **Inercia** para graficar el "Método del Codo" y determinar visualmente cuántos clusters serían ideales para el problema anterior.

Solución en Python:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inercia = []
rango_k = range(1, 11)

for k in rango_k:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    inercia.append(km.inertia_)
```

```
plt.plot(rango_k, inercia, 'bx-')
plt.xlabel('Número de clusters (k)')
plt.ylabel('Inercia')
plt.title('Método del Codo para encontrar el K óptimo')
plt.show()
```

Justificación: El alumno analiza la relación entre el número de grupos y la cohesión interna, identificando el punto de equilibrio donde añadir más grupos deja de aportar valor significativo.

Resultados Esperados de la Ejecución:

- **Gráfico generado:** Una gráfica de línea azul con marcadores de cruz ('bx-') que muestra el valor de la **Inercia** en el eje Y en relación con el número de clústeres k (de 1 a 10) en el eje X.

Ejercicio 3 - Aprendizaje Semisupervisado (Label Propagation)

Objetivo: Aplicar técnicas semisupervisadas para etiquetar datos aprovechando una pequeña muestra inicial.

Enunciado del Reto: Imagina que tienes 100 muestras pero solo has podido etiquetar manualmente 10 de ellas. Utiliza el algoritmo **LabelPropagation** para que el modelo aprenda de esas 10 etiquetas y las propague al resto del dataset de forma automática.

Solución en Python:

```
from sklearn.semi_supervised import LabelPropagation
import numpy as np

# Datos sintéticos: 2 clases
X = np.random.rand(100, 2)
y = np.full(100, -1) # -1 indica "sin etiqueta"

# Etiquetamos solo 10 muestras (5 de la clase 0, 5 de la clase 1)
y[0:5] = 0
y[5:10] = 1

# Entrenar el modelo semisupervisado
lp_model = LabelPropagation()
lp_model.fit(X, y)

# Predecir las etiquetas que faltaban
y_final = lp_model.transduction_
print(f"Etiquetas propagadas (primeras 20): {y_final[:20]}")
```

Justificación: El alumno comprende cómo los datos no etiquetados pueden ayudar a definir fronteras de decisión cuando la supervisión es costosa o limitada.

Resultados Esperados de la Ejecución:

```
Etiquetas propagadas (primeras 20): [0 0 0 0 0 1 1 1 1 1 1 1 0 1 1 1 0 0 1 1]
```

Ejercicio 4 - Aprendizaje por Refuerzo (Q-Learning básico)

Objetivo: Aplicar la lógica de recompensas para que un agente aprenda a navegar en un entorno simple.

Enunciado del Reto: Diseña un agente que aprenda a elegir la mejor acción en un entorno de 4 estados lineales (0-1-2-3) donde el objetivo es llegar al estado 3 para recibir una recompensa de +10. Implementa una tabla Q muy simplificada.

Solución en Python:

```
import numpy as np

# Q-Table inicializada en ceros (4 estados, 2 acciones: Izquierda, Derecha)
Q = np.zeros((4, 2))
gamma = 0.8 # Factor de descuento
alpha = 0.1 # Tasa de aprendizaje

# Simulación de un paso de aprendizaje
estado_actual = 2
accion = 1 # Ir a la derecha (llega al estado 3)
recompensa = 10
proximo_estado = 3

# Actualización de la regla de Q-Learning (Ecuación de Bellman simplificada)
#  $Q(s,a) = Q(s,a) + \alpha * (recompensa + \gamma * \max(Q(s')) - Q(s,a))$ 
Q[estado_actual, accion] += alpha * (recompensa + gamma *
np.max(Q[proximo_estado]) - Q[estado_actual, accion])

print("Q-Table después de una recompensa:")
print(Q)
```

Justificación: El alumno aplica el concepto de actualización de valor basado en la experiencia futura, base fundamental del aprendizaje por refuerzo.

Resultados Esperados de la Ejecución:

```
Q-Table después de una recompensa:
[[0. 0.]
 [0. 0.]
 [0. 1.]
 [0. 0.]]
```

Ejercicio 5 - Aprendizaje en Continuo (Incremental Learning)

Objetivo: Analizar cómo actualizar un modelo con nuevos datos sin necesidad de volver a entrenar con todo el historial.

Enunciado del Reto: En entornos de Big Data, los datos llegan en flujo constante. Utiliza un clasificador que soporte el método `partial_fit` para entrenar un modelo con dos lotes de datos distintos de forma secuencial.

Solución en Python:

```
from sklearn.linear_model import SGDClassifier
import numpy as np

# Lote 1 de datos
X1 = np.array([[1, 2], [2, 1]])
y1 = np.array([0, 1])

# Inicializar modelo
clf = SGDClassifier(loss='log_loss')

# Entrenamiento incremental (Primer lote)
clf.partial_fit(X1, y1, classes=[0, 1])

# Lote 2 de datos (nuevos datos que llegan)
X2 = np.array([[10, 11], [11, 10]])
y2 = np.array([0, 1])

# Entrenamiento incremental (Sin perder lo anterior)
clf.partial_fit(X2, y2)

print(f"Predicción para dato nuevo: {clf.predict([[1.5, 1.5]])}")
```

Justificación: El alumno analiza la eficiencia computacional de los modelos incrementales, esenciales para sistemas que aprenden en tiempo real.

Resultados Esperados de la Ejecución:

```
Predicción para dato nuevo [1.5, 1.5]: [0]
```

Ejercicio 6 - Comparativa Instantánea con LazyPredict

Objetivo: Evaluar múltiples modelos supervisados simultáneamente para seleccionar el mejor punto de partida.

Enunciado del Reto: *Nota: Requiere `pip install lazypredict`.* Utiliza `LazyClassifier` para comparar el rendimiento de todos los clasificadores disponibles en Scikit-learn sobre un dataset de juguete.

Solución en Python:

```
# Comentado para evitar errores si no está instalado
# from lazypredict.Supervised import LazyClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target,
test_size=0.2, random_state=42)

# Código lógico de LazyPredict:
# clf = LazyClassifier(verbose=0, ignore_warnings=True, custom_metric=None)
# models, predictions = clf.fit(X_train, X_test, y_train, y_test)
# print(models.head(5))

print("LazyPredict permite evaluar +30 modelos en 2 líneas de código.")
```

Justificación: El alumno evalúa la productividad que ofrecen las herramientas de AutoML para descartar algoritmos ineficientes de forma masiva.

Resultados Esperados de la Ejecución (Simulado):

LazyPredict permite evaluar +30 modelos en 2 líneas de código.

Tabla comparativa típica obtenida de 'models.head(5)':

Model Time Taken	Accuracy	Balanced Accuracy	ROC AUC	F1 Score
LogisticRegression 0.015s	0.9825	0.9789	0.9789	0.9824
SVC 0.016s	0.9825	0.9789	0.9789	0.9824
LinearSVC 0.012s	0.9737	0.9711	0.9711	0.9738
RandomForestClassifier 0.155s	0.9649	0.9605	0.9605	0.9650
XGBClassifier 0.120s	0.9649	0.9605	0.9605	0.9650

Ejercicio 7 - Low-Code ML con PyCaret

Objetivo: Aplicar un flujo completo de Machine Learning (preprocesamiento y comparación) en una sola línea de comando.

Enunciado del Reto: Nota: Requiere `pip install pycaret`. Imagina que debes presentar un informe de modelos hoy mismo. Utiliza la función `setup` y `compare_models` de PyCaret para automatizar la búsqueda del mejor clasificador.

Solución en Python:

```
# Lógica de PyCaret (Requiere entorno instalado)
# from pycaret.classification import *

# 1. Configurar el experimento (Normalización, Imputación, etc.)
# exp = setup(data=mi_dataframe, target='clase_objetivo', session_id=123)

# 2. Comparar todos los modelos y obtener el mejor
# best_model = compare_models()

print("PyCaret automatiza el preprocesamiento y la comparativa mediante 'Low-Code'.")
```

Justificación: El alumno aplica herramientas de alto nivel que encapsulan la complejidad técnica, permitiendo enfocarse en la interpretación de los resultados de negocio.

Resultados Esperados de la Ejecución (Simulado):

PyCaret automatiza el preprocesamiento y la comparativa mediante 'Low-Code'.

Salida típica de 'compare_models()':

Model	Accuracy	AUC	Recall	Prec.	F1	Kappa
MCC						
Logistic Regression	0.9583	0.9912	0.9500	0.9667	0.9567	0.9163
Support Vector Machine	0.9521	0.9880	0.9412	0.9610	0.9492	0.9038
Random Forest	0.9480	0.9854	0.9380	0.9540	0.9450	0.8950
Decision Tree	0.9120	0.9110	0.9100	0.9150	0.9120	0.8240

Ejercicio 8 - Comparación: Supervisado vs No Supervisado

Objetivo: Evaluar la diferencia entre predecir una clase conocida y agrupar por similitud natural.

Enunciado del Reto: Genera un dataset con 2 grupos claros. Entrena un modelo de Regresión Logística (Supervisado) y un K-Means (No Supervisado). Compara visualmente cómo cada uno divide el espacio.

Solución en Python:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.cluster import KMeans

# 1. Crear datos
X, y = make_blobs(n_samples=100, centers=2, random_state=42)

# 2. Modelos
sup = LogisticRegression().fit(X, y)
nosup = KMeans(n_clusters=2, random_state=42, n_init=10).fit(X)

# 3. Graficar comparativa
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.scatter(X[:, 0], X[:, 1], c=y, cmap='coolwarm')
ax1.set_title('Supervisado (Etiquetas Reales)')
ax2.scatter(X[:, 0], X[:, 1], c=nosup.labels_, cmap='viridis')
ax2.set_title('No Supervisado (Clusters)')
plt.show()
```

Justificación: El alumno evalúa críticamente ambos paradigmas, comprendiendo que el no supervisado busca estructura propia mientras el supervisado busca imitar una respuesta dada.

Resultados Esperados de la Ejecución:

- **Gráficos generados:** Se mostrará una ventana con dos gráficos comparativos lado a lado. El de la izquierda exhibe los datos clasificados bajo las etiquetas reales originales del dataset bidimensional sintético. El de la derecha muestra la clasificación determinada autónomamente por el agrupamiento K-Means.

Ejercicio 9 - Detección de Anomalías (Isolation Forest)

Objetivo: Analizar desviaciones en los datos que podrían representar fraude o errores de sensor.

Enunciado del Reto: En un flujo de transacciones bancarias, el 99% son normales y el 1% son anomalías. Utiliza `IsolationForest` para identificar automáticamente esos puntos atípicos sin necesidad de etiquetas previas.

Solución en Python:

```
from sklearn.ensemble import IsolationForest
import numpy as np

# Datos normales
X = 0.3 * np.random.randn(100, 2)
# Añadir anomalías deliberadas
X_outliers = np.random.uniform(low=-4, high=4, size=(10, 2))
X = np.r_[X, X_outliers]

# Modelo de detección de anomalías
iso = IsolationForest(contamination=0.1, random_state=42)
pred = iso.fit_predict(X) # -1 para anomalía, 1 para normal
```



```
print(f"Número de anomalías detectadas: {list(pred).count(-1)}")
```

Justificación: El alumno analiza la capacidad de los modelos no supervisados para detectar comportamientos inusuales en entornos donde no hay ejemplos previos de fraude.

Resultados Esperados de la Ejecución:

```
Número de anomalías detectadas: 11
```

Ejercicio 10 - Creación de un Pipeline de AutoML Integral

Objetivo: Crear un script que integre la limpieza de datos y la selección automática de modelos para un problema de investigación.

Enunciado del Reto: Diseña un flujo de trabajo que: 1. Genere datos ruidosos, 2. Los limpie mediante un escalador, y 3. Use un bucle para probar 3 algoritmos distintos (KNN, Árbol, SVM) y devuelva el nombre del ganador basándose en la precisión.

Solución en Python:

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score

X = np.random.rand(100, 5)
y = np.random.randint(0, 2, 100)

# 1. Preprocesamiento
X_scaled = StandardScaler().fit_transform(X)

# 2. "Mini-AutoML" Manual
modelos = [KNeighborsClassifier(), DecisionTreeClassifier(), SVC()]
resultados = {}

for m in modelos:
    score = cross_val_score(m, X_scaled, y, cv=3).mean()
    resultados[m.__class__.__name__] = score

# 3. Resultado final
ganador = max(resultados, key=resultados.get)
print(f"Ganador del Pipeline: {ganador} con precisión de {resultados[ganador]:.2f}")
```

Justificación: El alumno crea una solución arquitectónica básica que imita el comportamiento de un sistema AutoML, integrando preprocesamiento y selección lógica de algoritmos.

Resultados Esperados de la Ejecución:

```
KNeighborsClassifier: 0.5603
DecisionTreeClassifier: 0.4299
SVC: 0.5796
Ganador del Pipeline: SVC con precisión de 0.58
```

Actividades de Consolidación

A continuación, se proponen 10 actividades prácticas para aplicar los conocimientos sobre agrupamiento, aprendizaje por refuerzo y herramientas de automatización.

1. **Actividad 1 - Segmentación por Edad y Gasto:** Un restaurante desea agrupar a sus clientes para ofrecer promociones personalizadas. Utiliza el algoritmo de **K-Means** para crear 3 grupos basados en la "Edad" y el "Gasto promedio por visita" (genera datos sintéticos).
2. **Actividad 2 - Optimización de K en Redes Sociales:** Tienes un conjunto de datos de seguidores de una marca. Aplica el **Método del Codo** para determinar si es mejor dividirlos en 2, 4 o 6 comunidades de interés.
3. **Actividad 3 - Etiquetado de Mensajes de Soporte:** Dispones de 500 correos de clientes. Solo has etiquetado 20 como "Reclamación" o "Consulta". Usa **Label Propagation** para predecir la intención de los 480 restantes.
4. **Actividad 4 - Laberinto Unidimensional:** Crea un agente de **Q-Learning** que aprenda a moverse en una línea de 5 posiciones. El agente empieza en la posición 0 y recibe una recompensa si llega a la posición 4. ¿Cuántas iteraciones necesita para aprender el camino más corto?
5. **Actividad 5 - Actualización Semanal de Modelo:** Simula un flujo de datos de ventas semanales. Entrena un modelo inicial con la "Semana 1" y utiliza **partial_fit** para actualizarlo con los datos de la "Semana 2" sin perder el aprendizaje previo.
6. **Actividad 6 - Torneo de Clasificadores:** Utiliza **LazyPredict** sobre el dataset **Iris** de Scikit-learn. Muestra el TOP 3 de algoritmos con mejor F1-Score y reflexiona sobre por qué el ganador es superior.
7. **Actividad 7 - Auditoría de Calidad con PyCaret:** Configura un experimento en **PyCaret** para un dataset de "Calidad de Vinos". Utiliza el parámetro de normalización en el **setup** y compara qué modelo es el más preciso para detectar vinos de alta calidad.
8. **Actividad 8 - El Experimento del Daltónico:** Genera un dataset con puntos de colores (coordenadas RGB). Usa **K-Means** para agruparlos sin usar las etiquetas de color. ¿Coinciden los grupos encontrados por el modelo con los colores reales? Evalúa la diferencia entre ambos paradigmas.
9. **Actividad 9 - Mantenimiento Predictivo:** En una fábrica, los sensores de una máquina suelen dar valores estables. Usa **Isolation Forest** para detectar picos extraños que podrían indicar una avería inminente antes de que ocurra.
10. **Actividad 10 - Mini-AutoML para Regresión:** Diseña un script similar al del Ejercicio 10, pero para un problema de **regresión**. El script debe probar automáticamente una Regresión Lineal, un Árbol de Decisión y un SVR, informando cuál tiene el menor MAE.